

CSS Grid Layout

Building with Rows & Columns

Master grid track creation and sizing

Easy English • 50+ Examples • Ready to Use

Contents

1	Grid Track Basics	2
1.1	What are Grid Tracks?	2
1.2	Key Properties	2
2	Grid-Template-Columns	3
2.1	Fixed Width Columns	3
2.2	Flexible Columns with FR Units	3
2.3	Percentage and Other Units	4
3	Grid-Template-Rows	5
3.1	Fixed Height Rows	5
3.2	Flexible Rows	5
4	The repeat() Function	6
4.1	Basic Repeat	6
4.2	Different repeat() Patterns	7
5	MinMax Function	8
5.1	What is minmax()?	8
5.2	Common Patterns	8
6	Practical Examples	9
6.1	Example 1: Simple 3-Column Layout	9
6.2	Example 2: Sidebar + Main Content	10
6.3	Example 3: Responsive Grid	10
7	Summary	11

Chapter 1

Grid Track Basics

1.1 What are Grid Tracks?

Grid tracks are the rows and columns that make up your grid. They are defined using `grid-template-columns` and `grid-template-rows` properties. Each track is a space between two grid lines where items are placed.

1.2 Key Properties

- `grid-template-columns` - Defines column tracks
- `grid-template-rows` - Defines row tracks
- `repeat()` - Repeat track patterns
- `fr` - Fractional unit for flexible sizing
- `minmax()` - Set minimum and maximum sizes

Chapter 2

Grid-Template-Columns

2.1 Fixed Width Columns

Create columns with specific sizes:

CSS Code

```
.container {  
  display: grid;  
  grid-template-columns: 200px 300px 150px;  
  /* 3 columns: 200px, 300px, 150px */  
}
```

2.2 Flexible Columns with FR Units

Distribute available space proportionally:

CSS Code

```
/* Equal columns */
.container {
  grid-template-columns: 1fr 1fr 1fr;
  /* 3 equal flexible columns */
}

/* Proportional columns */
.container {
  grid-template-columns: 1fr 2fr 1fr;
  /* 1:2:1 ratio */
}

/* Mix fixed and flexible */
.container {
  grid-template-columns: 200px 1fr 200px;
  /* Fixed, flexible, fixed */
}
```

2.3 Percentage and Other Units

CSS Code

```
/* Percentages */
.container {
  grid-template-columns: 25% 50% 25%;
}

/* Em units */
.container {
  grid-template-columns: 10em 15em 10em;
}

/* Mixed units */
.container {
  grid-template-columns: 200px 1fr 20%;
}
```

Chapter 3

Grid-Template-Rows

3.1 Fixed Height Rows

CSS Code

```
.container {  
    display: grid;  
    grid-template-rows: 100px 200px 150px;  
    /* 3 rows with specific heights */  
}
```

3.2 Flexible Rows

CSS Code

```
/* Auto height based on content */  
.container {  
    grid-template-rows: auto auto auto;  
}  
  
/* Mixed sizing */  
.container {  
    grid-template-rows: 100px 1fr auto;  
}
```

Chapter 4

The repeat() Function

4.1 Basic Repeat

Avoid repetition when defining many tracks:

CSS Code

```
/* Without repeat - Verbose */
.container {
    grid-template-columns: 200px 200px 200px 200px 200px;
}

/* With repeat - Clean */
.container {
    grid-template-columns: repeat(5, 200px);
}

/* Multiple sizes */
.container {
    grid-template-columns: repeat(3, 200px 1fr);
    /* Creates 6 columns: 200px 1fr 200px 1fr 200px 1fr */
}
```

4.2 Different repeat() Patterns

CSS Code

```
/* Repeat flexible units */
.container {
  grid-template-columns: repeat(4, 1fr);
  /* 4 equal columns */
}

/* Repeat mixed pattern */
.container {
  grid-template-columns: repeat(3, 100px 2fr);
  /* Repeats "100px 2fr" three times */
}

/* Repeat with minmax */
.container {
  grid-template-columns: repeat(4, minmax(200px, 1fr));
}
```


Chapter 5

MinMax Function

5.1 What is minmax()?

The `minmax()` function sets minimum and maximum track sizes:

CSS Code

```
.container {  
  grid-template-columns: minmax(200px, 1fr) minmax(150px, 300px);  
  /* First column: at least 200px, grows to 1fr */  
  /* Second column: 150px minimum, 300px maximum */  
}
```

5.2 Common Patterns

CSS Code

```
/* Responsive columns */  
.container {  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  /* Columns of at least 250px, flexible */  
}  
  
/* Content-based minimum */  
.container {  
  grid-template-columns: minmax(min-content, 1fr);  
}  
  
/* Safe maximum */  
.container {  
  grid-template-columns: minmax(100px, 500px);  
}
```

Chapter 6

Practical Examples

6.1 Example 1: Simple 3-Column Layout

HTML

```
<div class="grid">
  <div>Column 1</div>
  <div>Column 2</div>
  <div>Column 3</div>
</div>
```

CSS

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
```

6.2 Example 2: Sidebar + Main Content

CSS

```
.layout {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  gap: 20px;  
  height: 100vh;  
}  
  
.sidebar {  
  background: #f0f0f0;  
}  
  
.main {  
  background: white;  
}
```

6.3 Example 3: Responsive Grid

CSS

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 15px;  
}  
  
@media (max-width: 1024px) {  
  .grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}  
  
@media (max-width: 768px) {  
  .grid {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (max-width: 480px) {  
  .grid {  
    grid-template-columns: 1fr;  
  }  
}
```

Chapter 7

Summary

Understanding how to create and size rows and columns is fundamental to CSS Grid. Master these concepts:

- Fixed sizes (px, em, rem)
- Flexible sizes (fr units)
- `repeat()` for patterns
- `minmax()` for responsive sizing
- Combining different unit types

These form the foundation for all other grid features!